

Experiment 1: Java Constructor and Class Practice

Experiment No: 01

Experiment Name: Java Class and Constructor Implementation

Task 1: Create a Car Class Using a Constructor Method

Task Name: Car Class with Constructor and displayCar() Method

Objective:

1. Create a Car class with attributes: brand, model, and price.
2. Implement a constructor to initialize the car's data.
3. Implement a displayCar() method to display the car's details.
4. Understand the use of constructors and object creation in Java.

Problem Analysis:

The task demonstrates object-oriented programming basics. Using a constructor ensures that every object of the class is initialized with proper values. The displayCar() method allows structured display of the object's information. This helps students understand how data and behavior are encapsulated in classes.

Input-Process-Output:

- **Input:** Car brand, model, price (via constructor when creating an object)
- **Process:** Constructor initializes the variables, displayCar() prints the data
- **Output:** Car details displayed on the console

Algorithm:

1. Define a Car class with three variables: brand, model, price.
2. Create a constructor to set values of these variables.
3. Define a displayCar() method to print the car's details.
4. In the main method, create objects of Car using the constructor.
5. Call displayCar() for each object.

Source Code:

```
CarDemo.java > ...
1  class Car {
2      String brand;
3      String model;
4      double price;
5
6      // Constructor
7      Car(String brand, String model, double price) {
8          this.brand = brand;
9          this.model = model;
10         this.price = price;
11     }
12
13     // Method to display car information
14     void displayCar() {
15         System.out.println("Car Brand: " + brand);
16         System.out.println("Car Model: " + model);
17         System.out.println("Car Price: $" + price);
18         System.out.println(x: "-----");
19     }
20 }
21
22 public class CarDemo {
23     Run | Debug
24     public static void main(String[] args) {
25         Car car1 = new Car(brand: "Toyota", model: "Corolla", price: 25000);
26         Car car2 = new Car(brand: "Honda", model: "Civic", price: 27000);
27
28         car1.displayCar();
29         car2.displayCar();
30     }
31 }
```

Output:

```
Car Brand: Toyota
Car Model: Corolla
Car Price: $25000.0
-----
Car Brand: Honda
Car Model: Civic
Car Price: $27000.0
-----
```

Discussion:

This task illustrates **object creation using constructors**. Every Car object is initialized with unique values at creation. Using displayCar() keeps data organized and demonstrates encapsulation. Constructors improve code efficiency and reduce errors compared to manually setting attributes after object creation.

Experiment 2:

Task 2: Create IDCard Class Using a Constructor Method

Task Name: IDCard Class with Constructor and showID() Method

Objective:

1. Create an IDCard class with attributes: name, id, department, institution.
2. Use a constructor to initialize the ID card information.
3. Implement a showID() method to display the ID card details.
4. Learn to encapsulate and display structured data in Java.

Problem Analysis (Hybrid Model):

This task teaches how to store and display structured information in objects. Using constructors ensures all ID cards are initialized correctly. showID() provides a clean way to output multiple pieces of related information, illustrating encapsulation and object-oriented design principles.

Input-Process-Output:

- **Input:** Name, ID, Department, Institution (via constructor)
- **Process:** Constructor initializes variables, showID() prints data
- **Output:** ID card details displayed on the console

Algorithm:

1. Define IDCard class with variables: name, id, department, institution.
2. Create a constructor to initialize these variables.
3. Define showID() method to print ID card details.
4. In the main method, create objects of IDCard.
5. Call showID() for each object.

Source Code:

```
IDCardDemo.java > IDCardDemo > main(String[])
1  class IDCard {
2      String name;
3      String id;
4      String department;
5      String institution;
6
7      // Constructor
8      IDCard(String name, String id, String department, String institution) {
9          this.name = name;
10         this.id = id;
11         this.department = department;
12         this.institution = institution;
13     }
14     // Method to display ID information
15     void showID() {
16         System.out.println("Name: " + name);
17         System.out.println("ID: " + id);
18         System.out.println("Department: " + department);
19         System.out.println("Institution: " + institution);
20         System.out.println(x: "-----");
21     }
22 }
23 public class IDCardDemo {
24     public static void main(String[] args) {
25         IDCard student1 = new IDCard(name: "Fahim Shariar", id: "E253071", department: "Computer and Communication
26         IDCard student2 = new IDCard(name: "Sharfuddin Faisal", id: "E253021", department: "Economics and Banking",
27         student1.showID();
28         student2.showID();
29     }
30 }
31
```

Output:

```
Name: Fahim Shariar
ID: E253071
Department: Computer and Communication Engineering
Institution: International Islamic University Chattogram
-----
Name: Sharfuddin Faisal
ID: E253021
Department: Economics and Banking
Institution: International Islamic University Chattogram
-----
```

Discussion:

The IDCard class shows how constructors simplify initialization and reduce repetitive code. Encapsulating data and behavior within a class demonstrates OOP principles, making programs modular, readable, and maintainable. Methods like showID() help separate logic for data display from data storage.